Linear Search

```
int search(int arr[], int n, int x)
{
    for(int i=0; i<n; i++)
        if (arr[i] == x)
            return i;
    return -1
}
int main(void) {
    int arr[] = {2, 3, 4, 10, 40};
    int x = 10;
    int n = sizeof(arr)/sizeof(arr[0])
    int result = search(arr, n, x);
    (result == -1) ? printf("Element is not present in array");
    : printf("Element is present at index %d", result);
    return 0;
}
```

Time complexity

worst case : O(n)

Space complexity

space complexity : O(1)

Binary Search

```
int binarysearch(int arr[], int n, int x)
{
    int low = 0;
    int high = n-1;
    while (low <= high)
    { int mid = low + (high - low)/2;
      if (arr[mid] == x)
          return mid;
      if (arr[mid] < x)
          low = mid + 1;
      else
          high = mid - 1;
    }
    return -1;
}
```

Time complexity	Space complexity
worst case: $O(\log n)$	$O(1)$

1. GCD of odd and even sums

```
int gcdofOddEven(int n)
{
    return n;
}
```

Example:
Input n = 4
Output: 4
Sum Odd = 1+3+5+7 = 16
Sum Even = 2+4+6+8 = 20
GCD = 4

2. Sum of GCD of Formal Pairs

```
int gcd(int a, int b)
{
    while (b)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
int comp(const void* a, const void* b)
{
    return(*(int*)a - *(int*)b);
}
```

```
long long gcdsum(int* nums, int numsSize){
int prefixGCD[numsSize];
int maxVal = nums[0];
for(int i=0; i<numsSize; i++)
{
    if (nums[i] > maxVal)
        maxVal = nums[i];
    PrefixGcd[i] = gcd(nums[i], maxVal);
}
sort(PrefixGCD, numsSize, sizeof(int),
     cmp);
long long sum=0;
for(int i=0; i<maxVal; i++)
    sum+ = gcd(PrefixGCD[i], PrefixGCD[numsSize
                                       -1]);
}
return sum;
}
```

3. Leet code 2447.

```
int gcd(int a, int b)
while (b)
{
    int t=b;
    b = a % b;
    a = t;
}
return a;
}
int subarrayGCD(int* nums, int numsSize,
                int k){
int count=0;
```

```
for (int i = 0; i < numsSize ; ++i ){
    int g = 0;
    for(int j = i; j < numsSize ; j++)
    {
        g = gcd(g, nums[j]);
        if (g == k)
            count++;
        else if (g < k)
            break;
    }
}
return count;
}
```

Input:
nums = [9, 3, 1, 2, 6, 3]
k = 3
output = 4

Explanation: Sub array of nums where 3 is the greatest common divisor of all the sub array's elements are

[9, 3, 1, 2, 6, 3]
[9, 3, 1, 2, 6, 3]
[9, 3, 1, 2, 6, 3]
[9, 3, 1, 2, 6, 3]

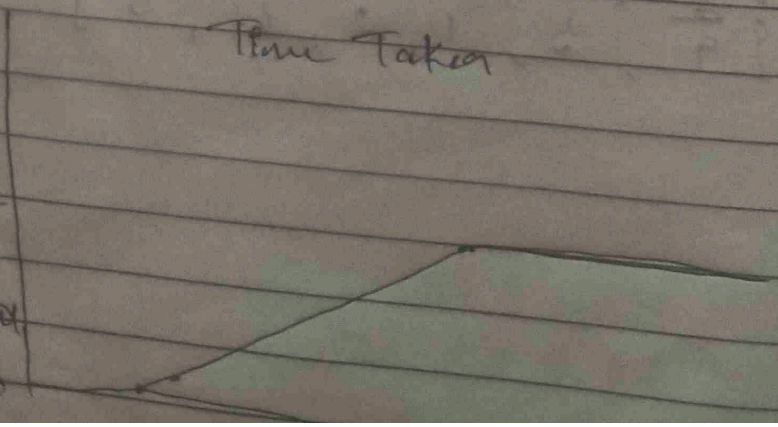
Merge Sort

```
merge sort (array)
    if length of array <= 1
        return array
    mid = length of array / 2

    left = array [0 ... mid]
    right = array [mid ... end]
    sorted left = mergesort (left)
    sorted right = merge sort (right)

    return merge (sorted left, sorted right)
```

Results

O/P:

Input Size	Time Taken
10000	0.00100
20000	0.00200
30000	0.00200
40000	0.00200
50000	0.00200



Quick sort.

Input - array → A
- starting index → low
- ending index → high

Output - sorted array A

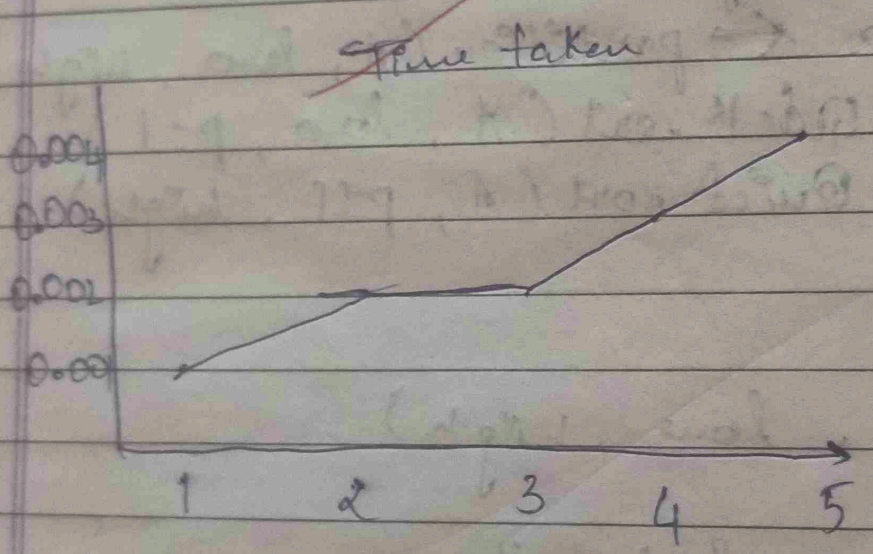
```
if low < high -
{   p ← partition (A, low, high)
    Quick sort (A, low, p-1)
    Quick sort (A, p+1, high)
}
```

Partition (A, low, high)

```
pivot ← A[low]
i ← low + 1
j ← high
while TRUE do -
    while i ≤ high AND A[i] ≤ pivot do -
        i ← i+1
    while A[j] > pivot do -
        j ← j-1
    if i < j then
        swap A[i] and A[j]
    else
        break
end while
swap A[low] and A[j]
return j
```

O/P:

Input Size	Time Taken
10000	0.00100
20000	0.00200
30000	0.00200
40000	0.00300
50000	0.00400



Prims Algorithm :

Purpose - To construct MST
Input - weighted connected graph
output - set of edges which forms MST

$V_T \leftarrow \{v_0\}$, $E_T \leftarrow \phi$
for $i \leftarrow 1$ to $|V|-1$ do
find an edge $e^* = (v^*, u^*)$ with minimum weight among all the edges $e = (v, u)$ such that v^* is in V_T and u^* is in $V-V_T$
$V_T \leftarrow V_T \cup \{u^*\}$
$E_T \leftarrow E_T \cup \{e^*\}$
end for
return E_T

Time Complexity = $|E| \log |V|$

O/P :

Edge	Weight
0-1	10
3-2	4
0-3	5

Total cost - 19

Kruskal's Algorithm:

Purpose - To construct MST
Input - weighted connected graph
output - set of edges which forms MST

Sort E in non-decreasing order of edge weight

$W(e_{i1}) \leq \cdots \leq W(e_{i|E|})$
$E_T \leftarrow \phi$ // edges of MST
count $\leftarrow 0$ // track no. of edges in MST
$k \leftarrow 0$ // initialize no. of processed edges
while count $< |V| - 1$
    $k \leftarrow k + 1$
    if $E_T \cup \{e_{ik}\}$ is acyclic
        $E_T \leftarrow E_T \cup \{e_{ik}\}$
        count $\leftarrow$ count $+1$
    end if
end while
return E_T

Time Complexity = $O(|E| \log |E|)$

O/P:

Edge	Weight
2 - 3	4
0 - 3	6
0 - 1	10

Total cost = 19

Topological sorting

```
#include <stdio.h>
#define MAX 100

int main() {
int n, i, j;
int adj[MAX][MAX], indegree[MAX] = {0};
int queue[MAX], front = 0, rear = -1;
int topo[MAX], count = 0;

printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter adj matrix: ");

for(i = 0; i < n; i++){
  for(j = 0; j < n; j++){
    if(adj[i][j] == 1){
      indegree[j]++;
    }
  }
}

for(i = 0; i < n; i++){
  if(indegree[i] == 0){
    queue[++rear] = i;
  }
}

while(front <= rear){
int v = queue[front++];
topo[count++] = v;

for(i = 0; i < n; i++){
```

```
        if (adj[v][i] == 1) {
            indegree[i]--;
            if (indegree[i] == 0) {
                queue[++rear] = i;
            }
        }
    }

    if (count != n) {
        printf("Graph has a cycle. Topological sorting not possible.\n");
    } else {
        printf("Topological order: ");
        for (i = 0; i < n; i++) {
            printf("%d ", topo[i]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter number of vertices: 4
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological order: 0 2 1 3
```

Johnson Trotter

```
Define Left = -1
Define Right = 1

Get Largest Mobile ( perm, dir, n ):
    largestMobile ← 0
    index ← -1

    for i from 0 to n-1 :
        neighbour ← i + dir[i]
        if neighbour within bounds and perm[i] > perm[neighbour]:
            if perm[i] > largestMobile:
                largestMobile ← perm[i]
                index ← i
    return index

function JohnsonTrotter(n):
    Initialize
    call printPermutation(perm, n)
    while (True):
        largestIndex ← getLargestMobile(perm, dir, n)
        if largestIndex == -1 , Brk
        swap ( largestIndex, dir[largestIndex])

        // swap elements
        Swap (perm[largestIndex], perm[swapIndex])

        // Swap direction
        Swap ( dir[largestIndex], dir[swapIndex])

        // Reverse directions of large elements
```

```
#include <stdio.h>
#define Left -1
#define Right 1

int getLargestMobile(int perm[], int dir[],
                     int n) {
    int largestMobile = 0, index = -1;
    for (int i = 0; i < n; i++) {
        int neighbour = i + dir[i];
        if (neighbour >= 0 && neighbour < n &&
            perm[i] > perm[neighbour]) {
            if (perm[i] > largestMobile) {
                largestMobile = perm[i];
                index = i;
            }
        }
    }
    return index;
}

void printPermutation(int perm[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d", perm[i]);
    printf("\n");
}

void johnsonTrotter(int n) {
    int perm[n], dir[n];
    for (int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = Left;
    }
    printPermutation(perm, n);
```

```
while (1)
{
    int largestIndex = getLargestMobile(perm, dir, n);
    if (largestIndex == -1) break;

    int swapIndex = largestIndex + dir[largestIndex];

    // Swap elements
    int temp = perm[largestIndex];
    perm[largestIndex] = perm[swapIndex];
    perm[swapIndex] = temp;

    // Swap directions
    int tempDir = dir[largestIndex];
    dir[largestIndex] = dir[swapIndex];
    dir[swapIndex] = tempDir;

    // Reverse direction of elements
    for (int i = 0; i < n; i++)
        if (perm[i] > perm[swapIndex])
            dir[i] = -dir[i];
    printPermutation(perm, n);
}
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("All permutations:\n");
    johnsonTrotter(n);
    return 0;
}
```

O/P:

Enter n: 3

All permutations:

1 2 3

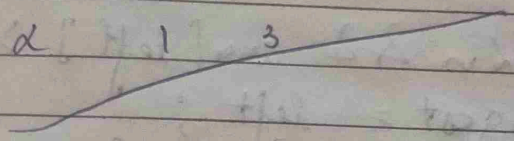
1 3 2

3 1 2

3 2 1

2 3 1

2 1 3



Leet code 316 :

```
char * removeDuplicateLetters(char* s) {
    int last[26], top = -1, n = strlen(s);
    bool visited[26] = {0};
    for (int i = 0 ; i < n ; i++)
        last[s[i] - 'a'] = i;

    char * st = malloc (n+1);
    for (int i = 0; i < n ; i++) {
        char c = s[i];
        if (vis[c - 'a']) continue;
        while (top >= 0 && c < st[top] &&
                i < last[st[top] -
        { vis[ st[top --] - 'a'] = 0;
        }
        st[++top] = c;
        vis[c - 'a'] = 1;
    }
    st[top++] = '\0';
    return st;
}
```

Input
s = "bcabc"

Output = "abc"

Expected = "abc"

Heap Sort:

```
#include <stdio.h>

void heapify (int arr[], int n, int i){
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify (arr, n, largest);
    }
}

void buildMaxHeap( int arr[], int n){
    for (int i = n/2 - 1; i >= 0; i--){
        heapify (arr, n, i);
    }
}

void heapSort (int arr[], int n){
    buildMAX Heap(arr, n);
```

```
for(int i = n-1 ; i >0 ; i--){
    int temp = arr[0];
    arr[0] = arr[i];
    arr[i] = temp;
    heapify(arr, i, 0);
}
}

void printArray(int arr[], int n){
    for(int i=0 ; i<n ; i++)
        printf("%d", arr[i]);
    printf("\n");
}

void main(){
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n], temp[n];
    printf("Enter elements: \n");
    for(int i=0 ; i<n ; i++){
        scanf("%d", &arr[i]);
        temp[i] = arr[i];
    }

.heapSort(arr, n).
 printf("\n Ascending order: \n");
 printArray(arr, n);
```

```
printf("\n Descending order : \n");
for(int i = n-1; i >= 0; i--)
    printf("%d ", arr[i]);
}
    printf("\n");
    return 0;
}
```

O/P:

Enter number of elements : 4
Enter elements
1 4 5 2

Ascending order :
1 2 4 5

Descending order :
5 4 2 1

Floyd's Algorithm

Inputs: V → set of weights (/distances)

Outputs: Shortest path between 2 points

```
//intermediate vertex
for K from 0 --> V
  // picking source vertex
  for i from 0 -> V
    // pick all destination vertex
    for j from 0 -> V
      if (dist[i][K] != ∞ && dist[K][j] != ∞)
      {
        dist[i][j] = min(dist[i][j],
                         dist[i][K] + dist[K][j]);
      }
```

```
#include <stdio.h>
#define MAX 10
#define INF 9999

void floyd(int graph[MAX][MAX], int n){
  int dist[MAX][MAX];
  for(int i=0; i<n; i++)
    for(int j=0; j<n; j++)
      dist[i][j] = graph[i][j];
  for(int K=0; K<n; K++)
    for(int i=0; i<n; i++)
      for(int j=0; j<n; j++)
        if(dist[i][K] + dist[K][j] <
           dist[i][j])
          dist[i][j] = dist[i][K]+dist[K]
          [j];
```

```
    printf("Shortest distance matrix:\n");
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
}

int main(){
    int n, graph[MAX][MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (use 9999 for INF):\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++)
            scanf("%d", &graph[i][j]);
    floyd(graph, n);
    return 0;
}
```

O/P: Enter number of vertices: 5
Enter adjacency matrix (use 9999 for INF):
0 4 9999 5 9999
9999 0 1 9999 6
2 9999 0 3 9999
9999 9999 1 0 2
1 9999 9999 4 0
Shortest distance matrix:
0 4 5 5 7
3 0 1 4 6
2 6 0 3 5
3 7 1 0 2
1 5 5 4 0

Leet Code 1334 :-

```
class Solution {
public:
    int findTheCity(int n, vector<vector<int>>& edges,
                    int distanceThreshold) {
    const int INF = 1e9;
    vector<vector<int>> dist(n, vector<int>(n, INF));
    for (int i=0 ; i<n; i++){
        dist[i][i] = 0;
    }
    for (auto &e : edges ){
        int u = e[0], v = e[1], w = e[2];
        dist[u][v] = w;
        dist[v][u] = w;
    }

    for (int k=0; k<n; k++){
        for (int i=0 ; i<n; i++){
            for (int j=0; j<n; j++){
                if (dist[i][k] < INF && dist[k][j] < INF
                {
                    dist[i][j] = min(dist[i][j], dist[i][k]+
                                     dist[k][j]);
                }
            }
        }
    }

    int minReachable = INT_MAX;
    int resultCity = -1;
    for (int i=0 ; i<n; i++){
        int count=0;
```

```
for (int j = 0; j < n; j++) {
    if (i != j && dist[i][j] <= distanceThreshold)
        count++;
}

if (count <= minReachable) {
    minReachable = count;
    resultCity = i;
}
}
return resultCity;
}
};
```

O/P:

n = 4

edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,

distance Threshold = 4

Output:

3

Expected:

3

chandra's

Leet Code 912 :

```
class Solution {
public:
    void merge(vector<int>& nums, int left,
               int mid, int right) {
        vector<int> temp;
        int i = left;
        int j = mid+1;
        while(i<=mid && j<=right){
            if(nums[i] <= nums[j]){
                temp.push_back(nums[i++]);
            } else {
                temp.pushback(nums[j++]);
            }
        }
        while(i<=mid) temp.push_back(nums[i++]);
        while(j<=right) temp.push_back(nums[j++]);

        for(int k=left; k<=right; k++){
            nums[k] = temp[k-left];
        }
    }
    void mergeSort(vector<int>& nums, int
                   left, int right) {
        if(left >= right) return;

        int mid = left + (right - left)/2;
        mergeSort(nums, left, mid);
        mergeSort(nums, mid+1, right);
        merge(nums, left, mid, right);
    }
```

```
vector<int> sortArray(vector<int>& nums) {
    mergeSort(nums, 0, nums.size()-1);
    return nums;
}
};
```

O/P:

Input -
nums = [5, 2, 3, 1]

Output -
nums = [1, 2, 3, 5]

Expected -
nums = [1, 2, 3, 5]

Radhe

Dynamic Programming (Knapsack % 0/1)

```
#include <stdio.h>
int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}
int knapsack(int W, int wt[], int val[], int n)
{
    int i, w;
    int K[n+1][W+1];

    for(i=0; i<=n; i++)
    {
        for(w=0; w<=W; w++)
        {
            if(i==0 || w==0)
            {
                K[i][w] = 0;
            }
            else if(wt[i-1] <= w)
            {
                K[i][w] = max(
                    val[i-1] + K[i-1][w-wt[i-1]],
                    K[i-1][w]);
            }
            else
            {
                K[i][w] = K[i-1][w];
            }
        }
    }
```

```
    return K[n][W];
}
void main()
{
    int n, W, i;
    printf("Enter no. of items");
    scanf("%d", &n);
    int val[n], wt[n];
    for(i=0; i<n; i++)
    {
        printf("\n Enter value of item %d:", i+1);
        scanf("%d", &val[i]);
        printf("Enter weight of item %d:", i+1);
        scanf("%d", &wt[i]);
    }
    printf("\n Enter capacity of knapsack: ");
    scanf("%d", &W);
    printf("Max profit = %d \n", knapsack(W, wt, val
}
```

O/P:

```
Enter number of items: 4
Enter value of item 1: 12
Enter wt of item 1: 1
Enter value of item 2: 20
Enter wt of item 2: 3
Enter value of item 3: 10
Enter wt of item 3: 1
Enter value of item 4: 15
Enter wt of item 4: 2
Max profit = 42
```

Fractional Knapsack

Pseudo code -

// Input - array of weights and corresponding array of profits + knapsack size
// Output - items to be included for max profit & profit.

```
void descendingPW ratio ( int val[ ], int wt[ ], int n)
{ for ( int i = 0; i < n-1; i++ ){
      for ( int j = 0; j < n-i-1; j++ )
        int ratio[i] = int val[i] / int wt[i];
        if( ratio[j] < ratio[j+1])
            swap (items[ j ], items[j+1]);
}

float fractionalKnapsack ( capacity )
{
    if (capacity >= items[i].weight ) {
        capacity -= items[i].weight;
        totalValue += item[i].value;
    }
    else {
        totalValue += items[i].value * ((float) capacity /
                                        items[i].weight
    }
return totalValue;
}
```

21/9/26

```
#include <stdio.h>

struct Item {
    int weight;
    int value;
    float ratio;
};
void swap (struct Item *a, struct Item *b){
    struct Item temp = *a;
    *a = *b;
    *b = temp;
}
void sortItems(struct Item items[], int n){
    for (int i=0; i<n-1; i++){
        for (int j=0; j<n-i-1; j++){
            if (items[j].ratio < items[j+1].ratio)
            { swap (&items[j], &items[j+1]); }
}}}

float fractionalKnapsack(int capacity, struct Item
                         items[], int n){
    sortItems(items, n);
    float totalValue = 0.0;
    for (int i=0; i<n; i++){
        if (capacity >= items[i].weight){
            capacity -= items[i].weight;
            totalValue += items[i].value;
        }
        else {
            totalValue += items[i].value * ((float)capacity/
                                            items[i].weight);
            break;
    }}
    return totalValue;
}
```

```
Void main ( ) {
    int n, capacity ;
    scanf (" %d", &n) ;
    struct Item items [n] ;

    for (int i = 0 ; i < n ; i++ ) {
        printf (" Enter value & weight of
                item %d : " , i+1 ) ;
        scanf (" %d %d ", & items[i].value,
                & items[i].weight ) ;
        items [i].ratio = (float) items[i].value /
                items [i].weight ;
    }
    scanf (" %d" , & capacity ) ;
    float maxValue = fractionalKnapsack (capacity,
                items, n ) ;
    printf (" Maximum value in Knapsack = %.2f\n"
                , maxValue ) ;
}
```

O/P : Enter number of items : 3
Enter value and weight of item 1 : 12
1
Enter value and weight of item 2 : 23
4
Enter value and weight of item 3 : 50
6
Enter knapsack capacity : 5
Maximum value in knapsack = 45.33

N-Queens -

```
#include <stdio.h>
#include <stdlib.h>

int N;
int *board;

int isSafe(int row, int col){
    for(int i=0; i<row; i++){
        if(board[i] == col || abs(board[i]-
                == abs(i-row)){
            return 0;
        }
    }
    return 1;
}

void printSolution(){
    for(int i=0; i<N; i++){
        for(int j=0; j<N; j++){
            if(board[i] == j)
                printf("Q");
            else
                printf(".");
        }
        printf("\n");
    }
    printf("\n");
}
```

```
void solveNQueens(int row) {
    if (row == N) {
        printSolution();
        return;
    }
    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solveNQueens(row + 1);
        }
    }
}

int main() {
    printf("Enter value of N: ");
    scanf("%d", &N);
    board = (int*) malloc(N * sizeof(int));
    printf("\n Solutions for %d-Queens
            Problem: \n\n", N);
    solveNQueens(0);
    free(board);
    return 0;
}
```

O/P: Enter the value of N: 4

Solutions for 4-Queens Problem:

. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q